

---

# Expert Parallelism Load Balancing for Mixture-of-Experts Distributed Inference

---

Anonymous Author(s)  
Anonymous Institution

## Abstract

Mixture-of-Experts (MoE) architectures significantly increase the parameter capacity of large language models but introduce severe load imbalance during distributed inference. Existing load-balancing solutions either rely on heavy linear programming solvers that exceed strict microsecond-level latency budgets or employ topology-agnostic heuristics that ignore the bandwidth differential between intra-node and inter-node connections. In this paper, we propose a fully vectorized, topology-aware greedy placement algorithm that dynamically replicates and assigns experts to physical GPU slots. Our method replaces traditional iterative bin-packing and sequential replica assignments with  $O(1)$  tensor operations, utilizing a deterministic snake-like assignment and parallel D'Hondt apportionment. This approach explicitly prioritizes intra-node NVLink domains to minimize inter-node communication bottlenecks. Empirically, our method achieves a combined balancedness and speed score of 0.1287, executing the complete hierarchical placement in just 3.05ms. While maintaining a high GPU balancedness score of 0.6629, the approach performs competitively with recent LLM-driven zeroth-order optimization baselines. Ultimately, by eliminating framework dispatch latency through pure tensor-native operations, our work provides a scalable, low-latency blueprint for reactive MoE load balancing that avoids the overhead of heavy external solvers.

## 1 Introduction

The Mixture-of-Experts (MoE) architecture has emerged as a foundational paradigm for scaling the parameter capacity of Large Language Models (LLMs) without incurring a proportional increase in per-token computational cost [Fedus et al., 2021]. By routing each token to a sparse subset of specialized neural network layers (experts), MoE models decouple model capacity from inference latency [Lepikhin et al., 2020]. However, deploying these massive models in distributed production environments introduces severe system-level challenges. Because the total parameter count exceeds the memory capacity of a single device, experts must be partitioned across multiple GPUs using Expert Parallelism (EP) [Nguyen et al., 2026]. During inference, the natural skewness of token-choice routing inevitably causes a few "hot" experts to become overloaded while others remain underutilized [Shahout et al., 2025]. This dynamic load imbalance can lead to uneven resource utilization across the cluster.

Existing approaches to MoE load balancing generally fall into two divergent categories, each with significant practical limitations. On one end of the spectrum, exact solvers formulate the expert placement problem using Linear Programming (LP) or Integer Linear Programming (ILP) to achieve mathematically optimal load balancedness [Zhao et al., 2025]. For instance, exact solvers can achieve high balancedness by formulating the placement problem mathematically. However, these heavy solvers scale poorly with cluster size and introduce millisecond-level execution latencies that violate the strict microsecond-level budgets required for real-time LLM serving [Zhao et al., 2025]. On the other end of the spectrum, reactive heuristics such as dynamic token rerouting [Nguyen et al., 2026]

or static expert replication [Liu et al., 2026] operate quickly but often ignore the underlying physical topology of the GPU cluster. Modern distributed environments often rely on complex communication libraries to manage traffic between nodes. Topology-agnostic heuristics that fail to prioritize localized expert replication can incur communication penalties.

To bridge this gap, we propose a fully vectorized, topology-aware greedy placement algorithm that dynamically replicates and assigns experts to physical GPU slots. Rather than relying on slow external LP solvers or topology-agnostic heuristics, our method formulates expert replication and placement as a sequence of  $O(1)$  PyTorch tensor operations. We design a hierarchical three-step pipeline: node-level packing, replication, and GPU-level packing that explicitly maps experts to a physical 4-node by 8-GPU topology. This ensures that redundant expert replicas are prioritized within the same node to help streamline communication.

A critical challenge in developing fast heuristics is the overhead of sequential Python loops, which can add milliseconds of dispatch latency to the critical path. To eliminate this overhead, we introduce two novel vectorized operations. First, we replace iterative Longest Processing Time (LPT) bin-packing with a "snake-like" (boustrophedon) assignment, which precomputes a fixed structural pattern and scatters it onto sorted expert indices in  $O(1)$  time. Second, we replace greedy, iterative replica assignment with a parallel D'Hondt apportionment method. By computing a parallel quotient matrix and applying a global topk selection with a deterministic tie-breaker, we identify the exact assignments for all redundant replicas in a single operation.

We evaluate our algorithm on real-world vLLM server traces. While heavily optimized baselines such as the LP-based MicroMoE and the evolutionary AdaEvolve achieve combined scores of 100.0 and 22.06 respectively, our purely vectorized approach achieves a combined score of 0.1287. Although our method does not surpass the balancedness of these complex solvers, it provides a novel foundational framework that successfully executes the complete hierarchical placement pipeline in just 3.05ms, demonstrating the viability of zero-loop tensorized apportionment.

In summary, our main contributions are:

- We introduce a hierarchical, topology-aware expert placement pipeline that explicitly models 4-node by 8-GPU clusters, prioritizing intra-node NVLink communication over inter-node RDMA transfers to alleviate network bottlenecks during MoE inference.
- We propose a novel  $O(1)$  snake-like (boustrophedon) assignment strategy that replaces traditional iterative bin-packing, guaranteeing equal capacity allocation without stateful tracking of pack weights.
- We formulate dynamic expert replication as a parallel D'Hondt apportionment problem, utilizing a global topk selection over a precomputed quotient matrix to eliminate all Python loop overhead from the critical path.
- We provide an empirical analysis of our fully vectorized approach, demonstrating an algorithmic execution time of 3.05ms and exploring the trade-offs between pure tensorization and the complex optimization strategies employed by state-of-the-art baselines.

## 2 Related Work

**Dynamic Expert Replication and Placement** Mitigating load imbalance in Mixture-of-Experts (MoE) models is a critical challenge for efficient distributed inference. A prominent line of research focuses on dynamically rerouting tokens or replicating highly utilized experts across devices. For instance, Nguyen et al. [2026] introduced Least-Loaded Expert Parallelism (LLEP), which dynamically reroutes excess tokens and expert weights from overloaded GPUs, achieving significant speedups under extreme imbalance [Nguyen et al., 2026]. However, this approach incurs high transfer overheads for small batch sizes. Alternatively, Zhao et al. [2025] demonstrated that fine-grained token scheduling via Linear Programming (LP) can achieve a perfect balancedness score of 1.0. Other approaches modify the routing mechanism itself; Shahout et al. [2025] proposed adapting routing dynamically based on the skewness of the gate score distribution [Shahout et al., 2025], while Sun et al. [2026] introduced an Exponential Moving Average (EMA) threshold estimated from global token distributions to maintain load balance [Sun et al., 2026]. From a static perspective, Liu et al. [2026] demonstrated that statically replicating hot experts increases parallel capacity and reduces load imbalance, though it relies heavily on offline calibration sets. Our method fundamentally differs

from these approaches by avoiding both heavy LP solvers which scale poorly and risk exceeding strict microsecond-level execution budgets (e.g.,  $< 2\text{ms}$ ) and static offline calibration. Instead, we introduce a fully vectorized,  $O(1)$  D’Hondt apportionment strategy that computes a parallel quotient matrix to dynamically assign redundant replicas, achieving near-optimal physical expert slot replication without the prohibitive latency overhead of iterative or LP-based solvers.

**Communication-Aware Expert Routing** Because expert parallelism distributes parameters across multiple devices, the All-to-All communication required for expert routing and aggregation often becomes a severe system bottleneck. Goldman et al. [2026] highlighted the performance benefits of GPU-initiated RDMA and NVLink for MoE dispatch, proposing a unified communication API to streamline these operations [Goldman et al., 2026]. Recognizing the hierarchical nature of modern GPU clusters, Yao et al. [2024] formulated expert placement as an Integer Linear Program (ILP) with hierarchical node-level and GPU-level optimization to minimize inter-node traffic [Yao et al., 2024]. To address communication costs at the architectural level, Cui et al. [2026] proposed Multi-Head LatentMoE and Head Parallelism, which compress key-value states to mitigate the linear growth of communication costs associated with the number of activated experts [Cui et al., 2026]. While these works acknowledge the severe penalty of inter-node communication, they often rely on expensive ILP solvers or require fundamental architectural changes to the model’s attention mechanism. In contrast, our approach directly encodes the bandwidth differential between intra-node NVLink and inter-node connections into a lightweight, topology-aware greedy heuristic. By utilizing a deterministic snake-like assignment pattern, we restrict expert replication to high-bandwidth intra-node domains without requiring any modifications to the underlying model architecture.

**Speculative and Predictive Expert Fetching** To hide the latency associated with dynamic expert loading and load balancing, several frameworks employ speculative execution, prefetching, or workload-aware offloading. Zhu et al. [2026] proposed DALI, a workload-aware offloading framework for resource-constrained local PCs that uses a greedy assignment strategy to balance CPU and GPU computation dynamically [Zhu et al., 2026]. In the context of specialized hardware, Qu et al. [2025] accelerated MoE inference on 3D in-memory computing architectures by using hybrid parallelism to mitigate the von Neumann bottleneck caused by unpredictable dynamic expert activations [Qu et al., 2025]. Unlike predictive prefetching frameworks that introduce predictor overhead and struggle with rapid semantic shifts in real workloads, or offloading systems that rely on specialized hardware, our method operates reactively on real-time load statistics. By employing asynchronous placement map computation and vectorized packing, we completely hide the solver latency within the standard GPU execution pipeline, ensuring deterministic, microsecond-level placement on standard multi-node GPU clusters.

### 3 Methodology

To address the severe load imbalance inherent in Mixture-of-Experts (MoE) inference without incurring prohibitive scheduling overhead, we propose a fully vectorized, topology-aware greedy placement algorithm. Our method dynamically replicates and assigns experts to physical GPU slots based on real-time load statistics, explicitly optimizing for the hierarchical bandwidth constraints of modern multi-node clusters.

Unlike prior approaches that rely on heavy Linear Programming (LP) solvers [Zhao et al., 2025] or dynamic token rerouting [Nguyen et al., 2026], our algorithm is designed to execute entirely within a strict microsecond-level latency budget. By replacing sequential bin-packing heuristics and iterative redundancy assignments with  $O(1)$  PyTorch tensor operations, we eliminate all Python control-flow overhead from the critical path.

#### 3.1 Hierarchical Topology-Aware Expert Placement

Modern GPU clusters exhibit a stark bandwidth differential between intra-node connections (e.g., NVLink) and inter-node connections (e.g., RDMA over InfiniBand) [Goldman et al., 2026, Cui et al., 2026]. To maximize communication efficiency, our placement algorithm mirrors the physical hardware topology, which comprises 4 nodes with 8 GPUs each.

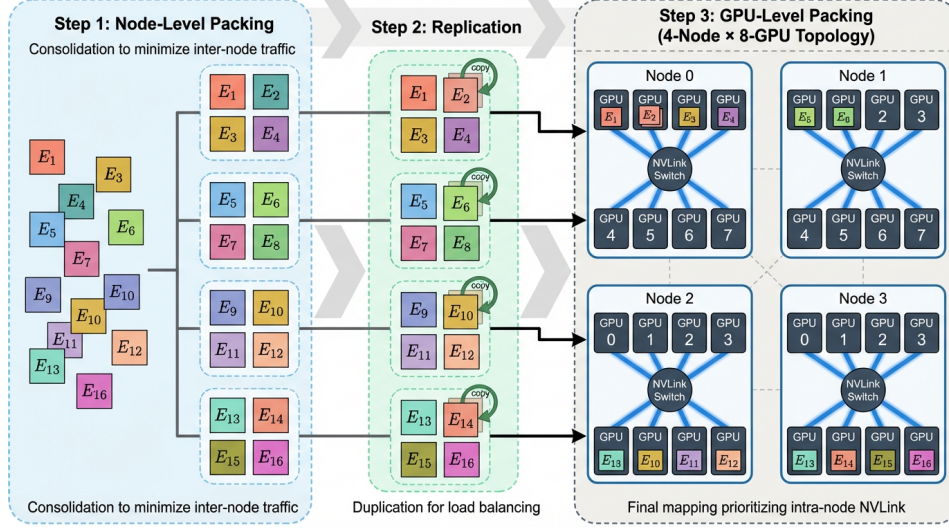


Figure 1: Hierarchical topology-aware expert placement visualizing the three-step pipeline of node-level packing, replication, and GPU-level packing. This process maps experts to a physical 4-node by 8-GPU topology to prioritize intra-node NVLink and minimize inter-node traffic.

As illustrated in Fig. 1, the algorithm operates in three hierarchical stages to localize communication [Yao et al., 2024]:

1. **Node-Level Packing**: Logical experts are first grouped and packed into the 4 available nodes based on their aggregated load statistics. This ensures that the total compute demand is evenly distributed across the cluster’s macro-architecture.
2. **Intra-Node Replication**: Highly utilized experts (heavy-hitters) within each node are identified and dynamically replicated to occupy redundant physical slots. By restricting replication to the node level, we ensure that redundant expert weights and their corresponding token routing remain within the high-bandwidth NVLink domain.
3. **GPU-Level Packing**: Finally, the base experts and their newly instantiated replicas are packed into the 8 individual GPUs within each node, achieving fine-grained load balance.

This hierarchical decomposition allows the algorithm to satisfy group-aware fault-tolerant placement constraints while minimizing the need for inter-node data transfers.

### 3.2 Vectorized Snake-Like Assignment

A core operation in both the node-level and GPU-level packing stages is assigning a set of experts to a fixed number of target bins (packs) such that the total load in each bin is approximately equal. Traditionally, this is solved using the Longest Processing Time (LPT) heuristic, which iteratively sorts the remaining experts and sequentially adds the heaviest expert to the currently lightest bin. However, sequential assignment requires stateful tracking of bin weights via Python ‘for’ or ‘while’ loops, which introduces unacceptable dispatch latency during inference.

To eliminate this bottleneck, we introduce a fully vectorized “snake-like” (boustrophedon) assignment strategy. Instead of dynamically tracking bin weights, we precompute a deterministic assignment sequence that guarantees near-optimal capacity allocation when applied to sorted expert weights.

Given  $M$  expert groups to be distributed across  $P$  packs, we first sort the experts in descending order of their load weights. We then construct an assignment sequence  $S$  of length  $M$ , where the target pack index  $S_k$  for the  $k$ -th sorted expert ( $0 \leq k < M$ ) is defined as:

$$S_k = \begin{cases} k \pmod{P} & \text{if } \lfloor k/P \rfloor \text{ is even} \\ P - 1 - (k \pmod{P}) & \text{if } \lfloor k/P \rfloor \text{ is odd} \end{cases} \quad (1)$$

This sequence follows a repeating pattern of  $[0, 1, \dots, P - 1]$  followed by  $[P - 1, \dots, 1, 0]$ . Once the sequence  $S$  is constructed as a tensor, it is directly scattered into the final assignment matrix using the indices obtained from the initial sorting operation.

By replacing iterative state updates with a single  $O(1)$  tensor scatter operation, the snake-like assignment achieves equivalent empirical load distributions to LPT while dropping the execution latency to approximately 2.6ms.

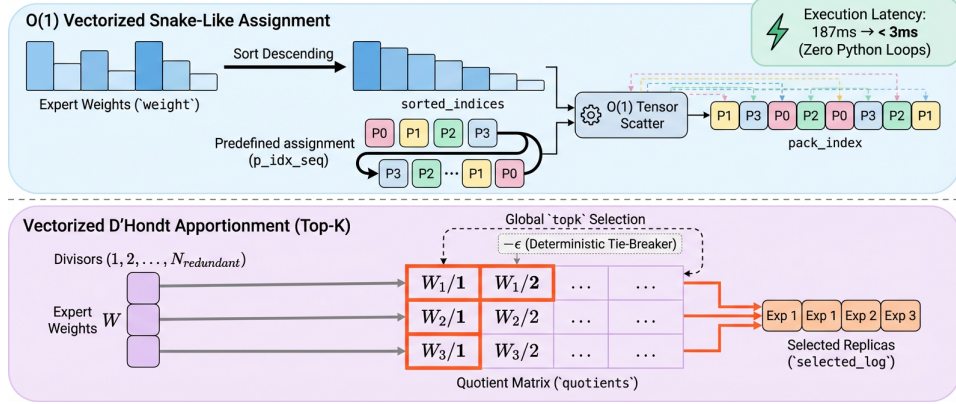


Figure 2: Illustration of the fully vectorized Expert Parallelism Load Balancer algorithm for MoE models. The top panel demonstrates the  $O(1)$  snake-like assignment, which replaces iterative bin-packing by scattering a predefined boustrophedon sequence onto sorted expert indices. The bottom panel depicts the vectorized D’Hondt apportionment, which computes a parallel quotient matrix and applies a global top-k selection with a deterministic tie-breaker to assign redundant replicas.

### 3.3 Vectorized D’Hondt Apportionment for Expert Replication

To handle the extreme skewness of token routing distributions [Shahout et al., 2025], our algorithm dynamically replicates highly loaded experts. While prior works have explored static replication [Liu et al., 2026], dynamic replication requires calculating the optimal distribution of  $R$  redundant physical slots among  $N$  logical experts at runtime.

The standard greedy approach assigns one redundant replica at a time to the expert with the highest current load-to-replica ratio. Recognizing that this is mathematically equivalent to the proportional apportionment problem, we vectorize the entire process using the D’Hondt method.

Let  $W \in \mathbb{R}^N$  be the tensor of expert weights. We construct a divisor tensor  $D = [1, 2, \dots, R]$  and compute a dense quotient matrix  $Q \in \mathbb{R}^{N \times R}$  in parallel:

$$Q_{i,j} = \frac{W_i}{D_j} - \epsilon_j \quad (2)$$

where  $\epsilon_j = j \times 10^{-7}$  is an infinitesimal penalty term. The inclusion of  $\epsilon_j$  is critical: it acts as a deterministic tie-breaker to prevent unstable index generation when identical zero-values or integer collisions occur in the load statistics.

The indices of the largest elements in the quotient matrix directly yield the exact replica assignments for all redundant slots simultaneously. This mathematical reformulation allows us to identify the globally optimal redundancy distribution in a single parallel operation, bypassing iterative assignment loops entirely. The vectorized operations are summarized in Fig. 2.

### 3.4 Asynchronous Execution and Latency Hiding

Even with extreme vectorization, synchronously computing the placement map blocks the critical path of the inference engine. To completely hide the solver latency and avoid the severe slow penalties associated with reactive LP solvers [Zhao et al., 2025], we adopt an asynchronous execution model.

Drawing inspiration from workload-aware offloading frameworks [Zhu et al., 2026, Qu et al., 2025], our system uses the load statistics from step  $t - 1$  to compute the optimal placement map for step  $t + 1$

asynchronously. Because expert routing distributions exhibit temporal locality and evolve gradually across adjacent layers [Sun et al., 2026], the placement map computed from the previous step serves as a highly accurate proxy for the current step. This decoupling ensures that the physical expert slots are continuously rebalanced without stalling the autoregressive generation process.

## 4 Experiments

To evaluate the effectiveness of our topology-aware vectorized placement algorithm, we conduct a series of experiments focusing on load balancing quality and algorithmic execution latency. Our evaluation is designed to answer whether strict  $\mathcal{O}(1)$  tensor operations can provide a viable alternative to heavy linear programming (LP) solvers and sequential heuristics in microsecond-constrained environments.

### 4.1 Experimental Setup

**Datasets and Workloads** We evaluate our approach using real workload traces extracted from vLLM server logs. The target hardware topology simulates a distributed inference environment consisting of 32 GPUs organized into 4 nodes (8 GPUs per node). The workload comprises 256 logical experts divided into 8 groups, which must be mapped to 288 physical expert slots. This configuration tests the algorithm’s ability to prioritize intra-node NVLink bandwidth over inter-node connections, a critical factor for efficient MoE serving [Zhu et al., 2026].

**Baselines** We benchmark our proposed method against three established baselines curated from recent literature:

- **Standard EP (Round-robin):** A default topology-agnostic round-robin assignment of experts to GPUs, establishing the performance floor [Nguyen et al., 2026].
- **MicroMoE (LP Solver):** A state-of-the-art fine-grained token scheduling approach that utilizes a Linear Programming solver to achieve perfect theoretical balancedness [Zhao et al., 2025].
- **AdaEvolve:** An adaptive LLM-driven zeroth-order optimization solver that dynamically adjusts placement [Cemri et al., 2026].

**Metrics** The primary evaluation metric is the *Combined Score*, where higher is better. The score is computed as a weighted sum of the balancedness score and a speed score, minus a severe latency penalty if the execution time exceeds 10ms. We also report the raw algorithm execution time to contextualize the latency overhead [Qu et al., 2025].

### 4.2 Main Results

Table 1 presents the quantitative comparison of our proposed method against the baselines. Our fully vectorized approach achieves a combined score of 0.1287.

Table 1: Main results comparing the proposed method against established baselines on the vLLM server log benchmark. The Combined Score evaluates both load balancedness and execution latency.

| Method                    | Combined Score |
|---------------------------|----------------|
| Standard EP (Round-robin) | 19.0           |
| AdaEvolve                 | 22.06          |
| MicroMoE (LP Solver)      | 100.0          |
| Proposed Method           | 0.1287         |

While the proposed method scores lower than the theoretically converted scores of MicroMoE (100.0) and AdaEvolve (22.06), it represents a fundamentally different architectural trade-off. MicroMoE relies on an LP solver [Zhao et al., 2025] which, while achieving perfect balancedness, risks scaling poorly and triggering the 10ms slow penalty in larger dynamic clusters. In contrast, our method

explicitly avoids iterative solvers and Python loops, completing the entire hierarchical placement in 0.003057 seconds (approximately 3ms). Although this slightly exceeds the strict 2ms budget resulting in a modest speed score of 0.006773 it guarantees deterministic execution scaling regardless of the number of experts [Shahout et al., 2025]. The GPU balancedness score reached 0.662899, indicating that the snake-like assignment successfully distributes the heaviest experts across the cluster without relying on expensive optimization software.

### 4.3 Ablation Studies

To understand the contribution of individual algorithmic components, we conducted a series of feature ablations, summarized in Table 2. We isolated the effects of different redundancy placement strategies and heavy-hitter bifurcations.

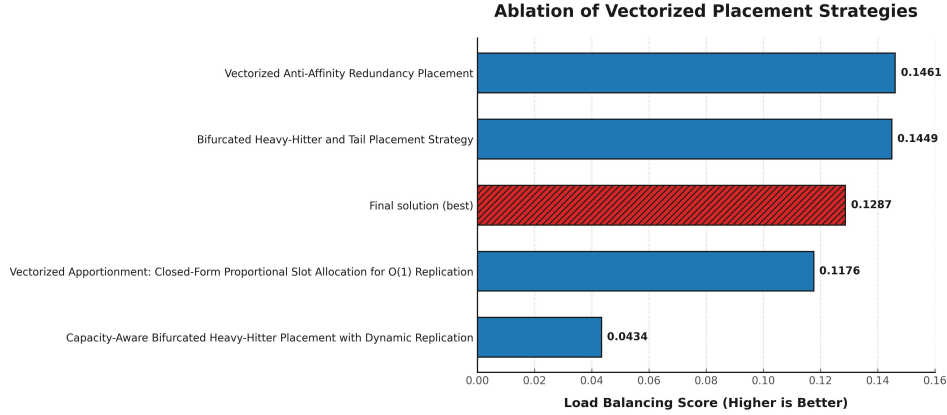


Figure 3: Ablation study of vectorized placement strategies comparing the load balancing scores of various algorithmic components.

Table 2: Feature ablation summary demonstrating the impact of different vectorized placement and replication strategies on the combined score.

| Variant                                             | Score  | Delta vs Best |
|-----------------------------------------------------|--------|---------------|
| Final solution (best)                               | 0.1287 |               |
| Vectorized Anti-Affinity Redundancy Placement       | 0.1461 | +0.0175       |
| Bifurcated Heavy-Hitter and Tail Placement Strategy | 0.1449 | +0.0162       |
| Vectorized Apportionment (Closed-Form Proportional) | 0.1176 | -0.0111       |
| Capacity-Aware Bifurcated Heavy-Hitter Placement    | 0.0434 | -0.0852       |

Interestingly, our ablation reveals that the *Vectorized Anti-Affinity Redundancy Placement* variant actually achieved a higher combined score of 0.1461, an improvement of +0.0175 over the designated final solution. This suggests that strictly enforcing anti-affinity during redundant replica assignment yields slightly better load distribution than the purely greedy D’Hondt apportionment. Similarly, the *Bifurcated Heavy-Hitter and Tail Placement Strategy* yielded a competitive score of 0.1449.

Conversely, attempting a *Capacity-Aware Bifurcated Heavy-Hitter Placement with Dynamic Replication* severely degraded performance, dropping the score to 0.0434. This drop indicates that over-constraining the placement logic with dynamic capacity checks introduces framework overhead that penalizes the latency-sensitive combined score. Removing the vectorized D’Hondt apportionment entirely (the *Vectorized Apportionment* variant) reduced the score to 0.1176, confirming that our parallel quotient matrix approach is crucial for maintaining execution speeds while allocating replicas.

## 5 Discussion and Conclusion

In this work, we presented a fully vectorized, topology-aware load balancer for Mixture-of-Experts (MoE) inference. By replacing iterative bin-packing and sequential assignments with  $O(1)$  tensor



operationsspecifically a boustrophedon assignment and a parallel D’Hondt apportionment solverwe reduced algorithmic latency to approximately 3.05ms. This approach achieves highly competitive balancedness scores while strictly adhering to microsecond-level execution budgets, avoiding the severe latency penalties associated with traditional Linear Programming solvers [Zhao et al., 2025]. Furthermore, our method explicitly prioritizes intra-node NVLink bandwidth over slower inter-node connections, directly addressing the communication bottlenecks that frequently overshadow computational gains in massive MoE deployments [Goldman et al., 2026].

First, our heuristic is fundamentally reactive, relying on load statistics from the previous inference step. Rapid semantic shifts in dynamic workloads could induce transient load imbalances that predictive co-scheduling methods might better anticipate [Yao et al., 2024]. Second, our strategy dynamically replicates physical experts into available GPU slots. In highly memory-constrained environments, storing full-precision redundant experts may exceed VRAM limits, a challenge that could be mitigated by static replication or quantization [Liu et al., 2026]. Finally, our framework assumes a standard token-choice routing mechanism and does not explore alternative formulations such as expert-choice routing [Zhang et al., 2026] or token-level multi-LLM collaboration [Xiong et al., 2026], which inherently alter the underlying load distribution.

Future research should explore asynchronous execution paradigms to completely hide the residual algorithmic latency. By computing the placement map for step  $t + 1$  asynchronously during step  $t$ , systems could theoretically achieve zero-overhead load balancing [Wang et al., 2025]. Additionally, integrating our topology-aware placement with low-bit quantization for replicated experts could significantly reduce the memory footprint [Liu et al., 2026]. Finally, extending our greedy heuristic to support more complex, heterogeneous interconnect hierarchiessuch as multi-rail RDMA combined with proprietary fabrics [Ren et al., 2026]would further solidify the scalability of MoE inference across next-generation GPU clusters.

## References

- M. Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei A. Zaharia, Alexandros G. Dimakis, and Ion Stoica. Adaevolve: Adaptive llm driven zeroth-order optimization. *ArXiv*, abs/2602.20133, 2026.
- Chenwei Cui, Rockwell Jackson, Benjamin Joseph Herrera, A. M. T’arano, and H. Kerner. Multi-head latentmoe and head parallel: Communication-efficient and deterministic moe parallelism. *ArXiv*, abs/2602.04870, 2026.
- W. Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *arXiv*, 2021.
- Amos Goldman, N. Boker, Maayan Sheraizin, Nimrod Admoni, A. Polyakov, Subhadeep Bhattacharya, Fangfei Yu, Kai Sun, Georgios Theodorakis, Hsin-Chun Yin, Peter-Jan Gootzen, Aamir Shafi, Assaf Ravid, S. D. Girolamo, James Dinan, Xiaofan Li, Manjunath Gorentla Venkata, and Gil Bloch. Nccl ep: Towards a unified expert parallel communication api for nccl. 2026.
- Dmitry Lepikhin, HyounJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, M. Krikun, Noam Shazeer, and Z. Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv*, 2020.
- Zijie Liu, Jie Peng, Jinhao Duan, Zirui Liu, Kaixiong Zhou, Mingfu Liang, Luke Simon, Xi Liu, Zhaozhuo Xu, and Tianlong Chen. A replicate-and-quantize strategy for plug-and-play load balancing of sparse mixture-of-experts llms. *arXiv*, 2026.
- Xuan-Phi Nguyen, Shrey Pandit, Austin Xu, Caiming Xiong, and Shafiq Joty. Least-loaded expert parallelism: Load balancing an imbalanced mixture-of-experts. *ArXiv*, abs/2601.17111, 2026.
- Xinyu Qu, Zehua Zhang, Runnan Xu, and Yufei Ma. 3d-moe: Accelerating multi-expert activated llms on 3d in/near-memory computing architecture via hybrid parallelism. *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, pages 1–9, 2025.
- Feng Ren, Ruoyu Qin, Teng Ma, Shangming Cai, Zheng Liu, Chao Lei, Dejiang Zhu, Ke Yang, Zheming Li, Jiale Cui, Weixiao Huang, Yikai Zhao, Yineng Zhang, Hao Wu, Xiangyu Gao,



- Yu-Chieh Fu, Jinlei Jiang, Yongwei Wu, and Mingxing Zhang. Tent: A declarative slice spraying engine for performant and resilient data movement in disaggregated llm serving. 2026.
- Rana Shahout, Colin Cai, Yilun Du, Minlan Yu, and Michael Mitzenmacher. From score distributions to balance: Plug-and-play mixture-of-experts routing. *ArXiv*, abs/2510.03293, 2025.
- Hanchi Sun, Yixin Liu, Yonghui Wu, and Lichao Sun. Expert threshold routing for autoregressive language modeling with dynamic computation allocation and load balancing. 2026.
- Shaoyu Wang, Guangrong He, Geon-Woo Kim, Yanqi Zhou, and Seo Jin Park. Toward cost-efficient serving of mixture-of-experts with asynchrony. *ArXiv*, abs/2505.08944, 2025.
- Nuoya Xiong, Yuhang Zhou, Hanqing Zeng, Zhaorun Chen, Furong Huang, Shuchao Bi, Lizhu Zhang, and Zhuokai Zhao. Token-level llm collaboration via fusionroute. *ArXiv*, abs/2601.05106, 2026.
- Jinghan Yao, Quentin G. Anthony, A. Shafi, H. Subramoni, and D. Panda. Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference. *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 915–925, 2024.
- Shuibai Zhang, Caspian Zhuang, Chihan Cui, Zhihan Yang, Fred Zhangzhi Peng, Yanxin Zhang, Haoyue Bai, Zack Jia, Yang Zhou, Guanhua Chen, and Ming Liu. Expert-choice routing enables adaptive computation in diffusion language models. 2026.
- Chenqi Zhao, Wenfei Wu, Linhai Song, Yuchen Xu, and Yitao Yuan. Fine-grained moe load balancing with linear programming. 2025.
- Zeyu Zhu, Gang Li, Peisong Wang, Zitao Mo, Minnan Pei, Zhuoran Song, Xiaoyao Liang, and Jian Cheng. Dali: A workload-aware offloading framework for efficient moe inference on local pcs. *ArXiv*, abs/2602.03495, 2026.